LAPORAN UJIAN TENGAH SEMESTER
KOMPUTASI PARALEL DAN TERDISTRIBUSI



Semester Ganjil 2026/2027
Judul Proyek : API Toko Suku Cadang Motor

Oleh:
Nama : Calvin Anjas Putra
NIM : 231220101
Kelas: TI31

Dosen Pengampu : Yulrio Brianorman, S.Si., M.T.

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS MUHAMMADIYAH PONTIANAK
2026

1. Deskripsi Aplikasi

Aplikasi yang dibangun adalah **API Toko Suku Cadang Motor** untuk membantu pemilik toko atau bengkel mengelola stok suku cadang dan mencatat pembelian. Resource utamanya adalah *products* (nama, kategori, merek, harga, stok) dan resource transaksinya adalah *transactions* (item_id, qty, total_harga, status). Data disimpan di SQLite dan diakses seluruhnya melalui aiosqlite secara asinkron.

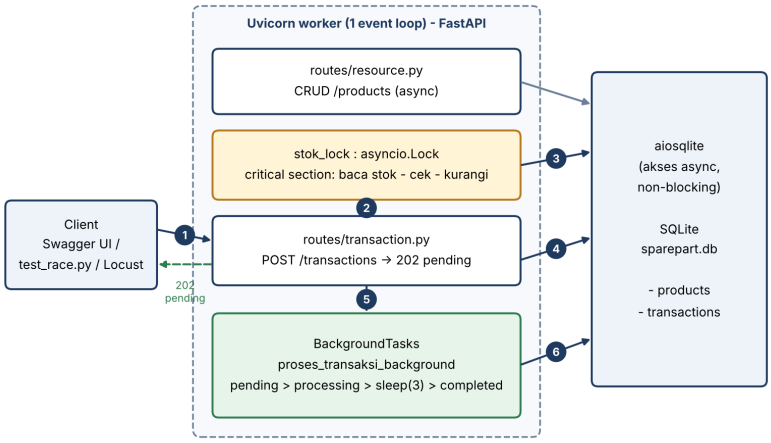
Fitur utama

- CRUD produk dengan validasi Pydantic dan paginasi `?skip=0&limit=10`.
- Pembuatan transaksi yang langsung dijawab **202** berstatus *pending*, lalu diproses di latar belakang (pending, processing, completed atau failed) menggunakan BackgroundTasks.
- Pengaman stok dengan `asyncio.Lock` agar stok tidak terjual melebihi persediaan; permintaan yang melebihi stok ditolak dengan **409 Conflict**.
- Dua versi fungsi pemotongan stok (tanpa lock dan dengan lock) yang dapat dipilih lewat variabel lingkungan `USE_LOCK` untuk keperluan pengujian.

Batasan sistem

- SQLite berupa satu berkas dengan satu penulis pada satu waktu, sehingga kurang cocok untuk beban tulis besar atau banyak server.
- `asyncio.Lock` hanya berlaku di dalam satu proses; pada multi-worker konsistensi stok antar-worker belum terjamin (dibahas pada bagian 6).
- Pemrosesan transaksi (sleep 3 detik) hanya simulasi dan tidak memakai antrean persisten. Bila server berhenti saat memproses, status transaksi dapat tertahan di *processing*.
- Belum ada autentikasi. Produk yang sudah memiliki riwayat transaksi tidak dapat dihapus (409).

2. Arsitektur dan Alur Data



Gambar 1. Diagram arsitektur: Client, Uvicorn dan FastAPI, asyncio.Lock, BackgroundTasks, serta SQLite melalui aiosqlite. Angka menunjukkan urutan alur transaksi.

Alur satu siklus transaksi (angka mengacu pada Gambar 1):

1. Client mengirim `POST /transactions` berisi item_id dan qty (panah 1). Pydantic memvalidasi body; data tidak valid ditolak dengan 422.
2. Route memeriksa keberadaan produk di SQLite lewat aiosqlite. Produk yang tidak terdaftar ditolak dengan 404.
3. Pemotongan stok masuk ke *critical section* `stok_lock` (panah 2 dan 3): baca stok, periksa kecukupan, lalu kurangi stok. Bila stok tidak cukup, dikembalikan 409 dan transaksi tidak dibuat.
4. Transaksi disimpan ke tabel *transactions* dengan status *pending* (panah 4).
5. Route mendaftarkan `proses_transaksi_background` melalui BackgroundTasks (panah 5) dan **langsung** mengembalikan 202 beserta id dan status *pending*. Client tidak menunggu proses selesai.

6. Setelah respons terkirim, pekerja latar belakang mengubah status menjadi *processing*, menunggu `await asyncio.sleep(3)`, lalu mengubahnya menjadi *completed* (panah 6). Bila terjadi galat, stok dikembalikan dan status menjadi *failed*.
7. Client memantau perubahan status melalui `GET /transactions/{id}` hingga bernilai *completed*.

3. Tabel Endpoint

Method	Path	Deskripsi fungsi	Status HTTP
GET	`/products?skip=0&limit=10`	Daftar produk dengan paginasi	200
GET	`/products/{id}`	Detail satu produk	200, 404
POST	`/products`	Tambah produk baru (validasi Pydantic)	201, 422
PUT	`/products/{id}`	Perbarui produk, mengembalikan data terbaru	200, 404, 422
DELETE	`/products/{id}`	Hapus permanen (hard delete); 409 bila produk sudah punya transaksi	200, 404, 409
POST	`/transactions`	Buat transaksi, langsung berstatus pending, diproses di latar belakang	202, 404, 409, 422
GET	`/transactions?status=`	Daftar transaksi, dapat difilter berdasarkan status	200, 422
GET	`/transactions/{id}`	Detail transaksi beserta status terkini	200, 404

Tabel 1. Daftar endpoint aplikasi.

4. Pemahaman Konsep: Concurrency dan Paralelisme

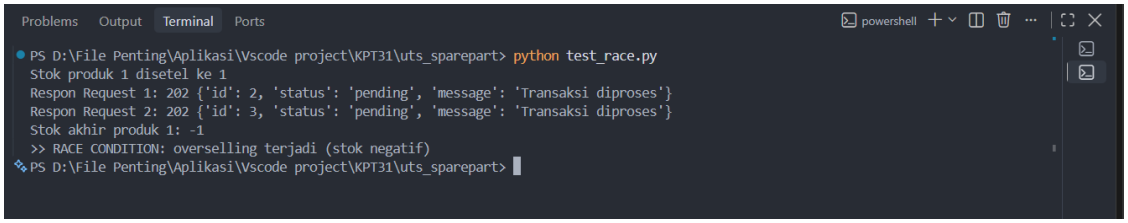
a. Bagian yang bersifat concurrent (I/O-bound). Akses SQLite melalui aiosqlite (membaca dan menulis produk dan transaksi) serta pekerja latar belakang yang menunggu `asyncio.sleep(3)` sebagai simulasi I/O adalah pekerjaan I/O-bound. Ketika satu coroutine menunggu (*await*), *event loop* menjalankan coroutine lain, sehingga banyak request dilayani bergantian dalam satu thread. Inilah concurrency: banyak tugas berjalan bergantian, bukan serentak di banyak inti CPU.

b. Bagian yang berpotensi membutuhkan paralelisme (CPU-bound). Pada aplikasi ini hampir tidak ada; perhitungan total_harga hanyalah perkalian sederhana. Paralelisme baru dibutuhkan bila sistem ditambah pekerjaan berat CPU, misalnya pembuatan laporan penjualan berukuran besar, agregasi data, atau hashing kata sandi. Pekerjaan seperti itu sebaiknya dijalankan dengan *multiprocessing* (misalnya ProcessPoolExecutor) dan tidak di dalam event loop, karena akan memblokir seluruh request lain.

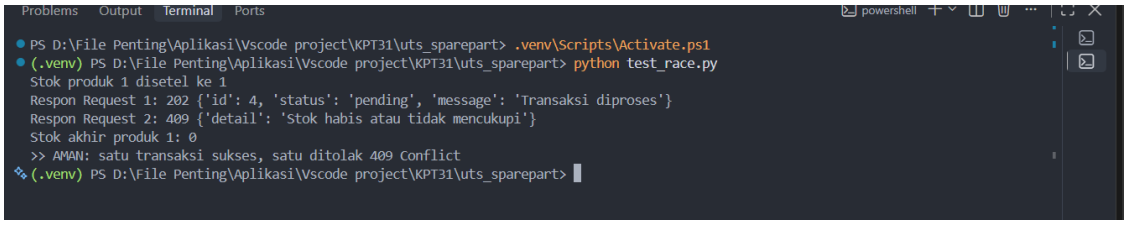
c. GIL dan multi-worker Uvicorn. Python GIL (*Global Interpreter Lock*) memastikan hanya satu thread yang mengeksekusi bytecode Python pada satu waktu di dalam satu proses. Akibatnya *threading* tidak mempercepat pekerjaan CPU-bound; thread hanya bermanfaat untuk I/O karena GIL dilepas saat menunggu I/O. Multi-worker Uvicorn (`--workers N`) menjalankan N **proses** terpisah, masing-masing dengan interpreter, GIL, dan event loop sendiri, sehingga dapat berjalan paralel di inti CPU yang berbeda. Konsekuensinya, memori tidak dibagi antar worker: `asyncio.Lock` dan variabel global pada satu worker tidak terlihat oleh worker lain, sehingga konsistensi data harus dijaga di level basis data.

5. Pengujian Race Condition (B3)

Pengujian dilakukan dengan server **1 worker** dan dua request `POST /transactions` yang dikirim serentak oleh `test_race.py` untuk produk yang stoknya 1.



Gambar 2. Tahap 1 (tanpa lock): kedua request berhasil (202) dan stok akhir menjadi -1 (overselling).



Gambar 3. Tahap 2 (dengan lock): satu request berhasil (202), satu ditolak 409 Conflict, stok akhir 0.

Analisis mekanisme lock. Pada tahap 1, setelah membaca stok program memanggil `await asyncio.sleep(0.01)`. Pada titik *await* itu event loop berpindah ke request kedua, sehingga keduanya membaca stok yang sama (1) sebelum ada yang mengurangi. Keduanya lolos pengecekan lalu sama-sama mengurangi stok, hasilnya -1. Event loop yang berjalan pada satu thread tidak otomatis aman, karena pergantian antar-coroutine terjadi di setiap *await*.

Pada tahap 2, blok `async with stok_lock` menjadikan urutan baca, cek, dan kurangi stok sebagai *critical section*: hanya satu coroutine yang boleh masuk, sedangkan yang lain ditangguhkan sampai gembok dilepas tanpa memblokir event loop. Request kedua baru membaca stok setelah dikurangi (0), pengecekan gagal, dan request ditolak dengan 409. Dengan demikian stok akhir 0 dan data tetap konsisten.

6. Hasil Benchmark dan Analisis Performa

Pengujian beban memakai Locust (`locustfile.py`) dan dijalankan otomatis oleh `benchmark.py` pada laptop Windows (Intel Core i3-1215U, RAM 16 GB) dengan Locust dan server berjalan di mesin yang sama. Seluruh request berhasil (0 gagal dari 150 request).

Skenario	Worker	Total Waktu (s)	Throughput (req/s)	Rata-rata Latensi (ms)
GET /products (100 request, 20 user)	1	0,405	246,94	69,29
GET /products (100 request, 20 user)	4	0,232	431,92	34,35
POST /transactions (50 request, 10 user)	1	3,692	13,54	678,06
POST /transactions (50 request, 10 user)	4	2,253	22,19	385,64

Tabel 2. Perbandingan 1 worker dan 4 worker.

GET /products. Dengan 4 worker, total waktu turun dari 0,405 s menjadi 0,232 s (42,7%), throughput naik 1,75 kali, dan latensi rata-rata turun 50,4%. Setiap request membuka koneksi aiosqlite dan menyusun JSON, yang memakan CPU Python. Pada 1 worker semua request berbagi satu GIL dan satu event loop sehingga antre; pada 4 worker beban terbagi ke beberapa proses di inti berbeda. Operasi baca juga relatif tidak saling menghalangi di SQLite (mode WAL).

POST /transactions. Total waktu turun dari 3,692 s menjadi 2,253 s (39,0%), throughput naik 1,64 kali, dan latensi rata-rata turun 43,1%. Throughput 1 worker sebesar 13,54 req/s berarti kira-kira 74 ms per transaksi, mendekati pemrosesan berurutan. Hal ini sejalan dengan pemotongan stok yang berada di dalam `stok_lock` (baca stok, jeda 10 ms, tulis stok), sehingga 10 request konkuren mengantre dan latensinya tinggi. Dengan 4 worker terdapat 4 lock yang independen, sehingga beberapa transaksi dapat diproses bersamaan. (Penjelasan ini adalah analisis, bukan hasil pengukuran terpisah.)

Mengapa tidak mencapai 4 kali lipat. Faktor yang kemungkinan membatasi: (1) Locust dan server berebut CPU pada laptop yang sama; (2) SQLite hanya mengizinkan satu penulis pada satu waktu sehingga penulisan dari banyak worker tetap bergantian; (3) jumlah request kecil (skenario GET selesai dalam

kurang dari 0,5 detik), sehingga waktu start dan variasi pengukuran berpengaruh besar. Angka di atas berasal dari satu kali pengujian dan dapat berbeda antar-percobaan.

Catatan konsistensi. Karena `asyncio.Lock` hanya berlaku di dalam satu proses, dua request yang diterima worker berbeda tidak saling menunggu, sehingga race condition dapat muncul kembali pada multi-worker. Benchmark ini tidak terdampak karena stok awal minimal 100 dan setiap transaksi hanya qty 1. Untuk penggunaan nyata, pengurangan stok sebaiknya dibuat atomik di level SQL (`UPDATE ... SET stok = stok - qty WHERE id = ? AND stok >= qty`) atau memakai penguncian terdistribusi pada tahap UAS.

7. Kesimpulan

- API Toko Suku Cadang Motor berhasil dibangun dengan FastAPI dan aiosqlite: CRUD lengkap, transaksi asinkron berstatus pending, processing, completed, serta pemisahan berkas sesuai ketentuan.
- Race condition pada pemotongan stok terbukti (stok -1) pada versi tanpa lock dan teratasi dengan `asyncio.Lock` (satu transaksi sukses, satu ditolak 409) pada server 1 worker.
- Penambahan worker dari 1 menjadi 4 meningkatkan throughput GET sebesar 1,75 kali dan POST sebesar 1,64 kali, namun lock per-proses menunjukkan perlunya penjagaan konsistensi di level basis data.

8. Referensi

1. FastAPI. Dokumentasi resmi. https://fastapi.tiangolo.com
2. Dokumentasi Python asyncio. https://docs.python.org/3/library/asyncio.html
3. aiosqlite. Dokumentasi. https://aiosqlite.omnilib.dev
4. Uvicorn. Dokumentasi. https://www.uvicorn.org
5. Locust. Dokumentasi. https://locust.io
6. Modul praktikum dan materi perkuliahan Komputasi Paralel dan Terdistribusi (Minggu 1 sampai 6).